

VMware & Hyper-V to KVM

End-to-End Automated Migration Pipeline

From Traditional Manual Steps to a Fully Automated Two-Tool Solution

hypersdk — Go

hyper2kvm — Python

Susant Sahani

LGPL-3.0 | Production Ready

Key Steps: Traditional VMware / Hyper-V → KVM Migration

VMware (vSphere / ESXi)

1. **Export VM** — OVF/OVA via ovftool or vSphere client
2. **Extract VMDK** — untar OVA, locate disk files
3. **Convert** — `qemu-img convert -f vmdk -O qcow2`
4. **Transfer** — SCP to KVM host
5. **Create VM** — `virt-manager / virt-install`
6. **Fix drivers, GRUB, fstab** — manual debugging
7. **Remove VMware Tools** — `uninstall open-vm-tools`

Hyper-V

1. **Shut down VM** — ensure data consistency
2. **Convert** — `qemu-img convert -f vhdx -O qcow2`
3. **Transfer** — SCP / WinSCP to KVM host
4. **Create VM** — map UEFI/Gen2 to OVMF
5. **Fix drivers** — install VirtIO, fix network
6. **Remove Hyper-V services** — integration components

Common Challenges (Both Platforms)

- **BSOD** Missing VirtIO drivers → boot failure
- **Network** VMXNet3 / Hyper-V vSwitch adapters gone
- **UEFI** Gen2 / EFI VMs need OVMF in KVM
- **fstab** Stale `/dev/sdX` names break mount
- **GRUB** Wrong disk controller in bootloader
- **VMware Tools** Conflicts with KVM guest agent
- **Snapshots** VMDK chains must be flattened first

Each step is manual, error-prone, and must be repeated per VM.

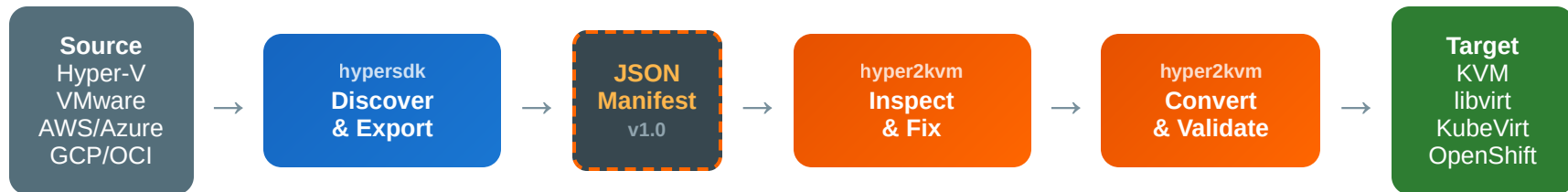
Traditional tools (`virt-v2v`, `qemu-img`, StarWind) only convert — they don't fix.

Our Solution: Two Tools, One Pipeline

hypersdk (Go) exports & discovers • **hyper2kvm** (Python) fixes & converts

Clean separation of concerns via a JSON manifest contract

Architecture: How They Work Together



hypersdk (Go daemon) handles the *messy outside world*:

- Multi-cloud provider abstraction (10 providers)
- VM discovery, export, transfer
- Job scheduling & REST API (67+ endpoints)
- Authentication, RBAC, audit logging
- Produces artifacts + manifest

hyper2kvm (Python) handles *deterministic operations*:

- Consumes manifest, never talks to providers
- Deep OS inspection via VMCraft engine
- Bootloader, fstab, initramfs, driver fixes
- Windows VirtIO injection + registry mods
- Validation & smoke testing

Contract principle: Same manifest → same outcome. Provider-agnostic. Deterministic. Resumable.

hypersdk — Multi-Cloud Export & Discovery

Supported Providers (10)

Provider	Status
vSphere (VMware)	Production
AWS (EC2)	Production
Azure	Production
GCP	Production
Hyper-V	Production
OCI (Oracle)	Production
OpenStack	Production
Alibaba Cloud	Production
Proxmox VE	Production
KubeVirt	Production

Components

hyperexport Interactive export	hypervisord REST API daemon	hyperctl CLI commander	hyperconvert Format converter
---	--	-------------------------------------	--

Key Capabilities

- **67+ REST endpoints** with WebSocket updates
- **Hexagonal architecture** — clean domain/ports/adapters
- **Job scheduling** — cron, dependencies, retry policies
- **Carbon-aware scheduling** — grid intensity optimization
- **Multi-format export** — OVF, OVA, VMDK, QCOW2, VHD
- **Incremental export** via Changed Block Tracking (CBT)
- **Security** — RBAC, API keys, audit logging, Vault

hyper2kvm — Fix, Convert & Validate Engine

480+

VMCraft APIs

35+

OS
Versions

5-

7x

Faster

380+

Tests

Automated Offline Fixes

- **Bootloader** — GRUB/GRUB2 repair for VirtIO
- **fstab** — UUID/PARTUUID stabilization
- **initramfs** — VirtIO driver injection
- **Network** — interface adaptation
- **LVM** — sandboxed activation (0.71s, 7x faster)

Windows Migration

- **VirtIO injection** — storage, network, balloon
- **Registry** — boot-critical driver registration
- **License** — OEM/Retail/MAK/KMS preservation
- **Active Directory** — domain rejoin support

VMCraft Engine Performance

Metric	libguestfs	VMCraft
Launch Time	9.7s	1.9s
Memory	280 MB	95 MB
System Calls	14,200	8,500

Code Example

```
from hyper2kvm.core.vmcraft import VMCraft

with VMCraft() as g:
    g.add_disk("/vms/server.qcow2")
    g.launch() # ~1.9 seconds

    os_info = g.inspect_os()
    print(f"OS: {os_info.product_name}")

    # Fix fstab, inject drivers, etc.
    g.write("/etc/motd", "Migrated!\n")
```

Deployment Targets

KVM

libvirt

KubeVirt

OpenShift

k3s

The Manifest Contract: How They Communicate

Communication is **file-based** via a versioned **JSON manifest**. hypersdk produces it, hyper2kvm consumes it.

Design Principles

- **Provider-agnostic** — hyper2kvm never talks to providers
- **Minimal requirements** — only disks[] + firmware hint
- **Deterministic** — same manifest → same outcome
- **Resumable** — checksums enable idempotent ops
- **Backward compatible** — within major version

Manifest Workflow

```
manifest-workflow/  
  to_be_processed/  # Drop manifests here  
  processing/       # Currently running  
  processed/        # Completed  
  failed/           # Errors
```

Manifest v1.0 (Simplified)

```
{  
  "manifest_version": "1.0",  
  "source": {  
    "provider": "hyperv",  
    "vm_name": "web-server-01",  
    "export_timestamp": "2026-03-10T..."  
  },  
  "vm": {  
    "cpu": 4, "memory_mb": 8192,  
    "firmware": "uefi"  
  },  
  "disks": [{  
    "path": "/exports/web-01.vhdx",  
    "format": "vhdx",  
    "size_bytes": 42949672960,  
    "controller": "scsi",  
    "checksum": "sha256:abc123..."  
  }],  
  "network": [{  
    "mac": "00:15:5d:01:02:03",  
    "type": "external"  
  }]  
}
```

Hyper-V Migration: Step by Step

Phase 1: Export (hypersdk)

```
# Start daemon
hypervisord --config config.yaml

# Export Hyper-V VM
hyperctl export \
  --provider hyperv \
  --vm "web-server-01" \
  --output /exports/

# Or schedule recurring
hyperctl schedules create \
  nightly-export \
  "0 2 * * *" \
  --vm web-server-01
```

Produces: VHDX files + JSON manifest with VM metadata, firmware type, network config

Phase 2: Fix & Convert (hyper2kvm)

```
# YAML config
cmd: local
vmdk: /exports/web-01.vhdx
output_dir: /kvm
to_output: web-01.qcow2
out_format: qcow2
fstab_mode: stabilize-all
regen_initramfs: true
no_grub: false

# For Windows VMs
virtio_iso: /path/to/virtio.iso

# Run
sudo h2kvmctl \
  --config migration.yaml
```

Automated: Disk convert + OS inspect + bootloader fix + driver inject + fstab stabilize

Phase 3: Deploy

```
# Option A: libvirt
sudo virsh define web-01.xml
sudo virsh start web-01

# Option B: Direct to K8s
sudo h2kvmctl \
  --config migration.yaml \
  --deploy-k8s \
  --k8s-namespace prod \
  --k8s-vm-name web-01 \
  --k8s-auto-start

# Verify
sudo virsh domifaddr web-01
nc -zv <IP> 22
```

Result: VM boots first time, every time. Network, drivers, storage all working.

VMware Migration: Step by Step

Phase 1: Export (hypersdk)

```
# Configure vSphere credentials
export GOVC_URL='https://vcenter/sdk'
export GOVC_USERNAME='admin@vsphere'
export GOVC_PASSWORD='password'

# Start daemon
hypervisord --config config.yaml

# Export from vSphere
hyperctl export \
  --provider vsphere \
  --vm /dc1/vm/web-server \
  --format ova \
  --output /exports/

# Batch export entire folder
hyperctl export \
  --provider vsphere \
  --folder /dc1/vm/prod/ \
  --concurrent 4
```

Produces: VMDK/OVA files + JSON manifest.
Supports CBT incremental export, snapshot management, resumable downloads.

Phase 2: Fix & Convert (hyper2kvm)

```
# YAML config
cmd: local
vmdk: /exports/web-server.vmdk
output_dir: /kvm
to_output: web-server.qcow2
out_format: qcow2
flatten: true
fstab_mode: stabilize-all
regen_initramfs: true
no_grub: false
compress: true
checksum: true

# Run
sudo h2kvmctl \
  --config migration.yaml
```

Handles: VMDK flatten (snapshot chains) +
VMware Tools cleanup + VirtIO inject +
GRUB repair + fstab stabilize

Phase 3: Deploy

```
# Option A: libvirt
sudo virsh define web-server.xml
sudo virsh start web-server

# Option B: Direct to K8s
sudo h2kvmctl \
  --config migration.yaml \
  --deploy-k8s \
  --k8s-namespace prod \
  --k8s-vm-name web-srv \
  --k8s-auto-start

# Option C: vSphere Python API
from hyper2kvm.vmware import \
    VMwareClient, ExportOptions
client = VMwareClient(
    vcenter_host='vcenter',
    username='admin@vsphere',
    password='pass'
)
await client.async_export_vm(
    vm_name='web-server',
    output_dir='/exports',
    export_mode='export'
)
```

Before vs After: Traditional vs Our Pipeline

Traditional (Manual)

1. Shut down VM manually
2. Run `qemu-img convert`
3. SCP file to KVM host
4. Create VM with `virt-manager`
5. Boot — **BSOD / panic**
6. Debug: install VirtIO drivers
7. Boot — **fstab error**
8. Debug: fix fstab entries
9. Boot — **GRUB error**
10. Debug: rebuild initramfs
11. Boot — **no network**
12. Debug: reconfigure network
13. Finally works (*hours later*)

~2-4 hours per VM • Error-prone • No batch

VS

hypersdk + hyper2kvm (Automated)

1. `hyperctl export --provider vsphere --vm web-01`
2. `h2kvmctl --config migration.yaml`
3. VM boots successfully **first time**

What happens automatically:

- **Export** VM discovery + disk download + manifest
- **Convert** VMDK/VHDX → QCOW2 with compression
- **Inspect** OS detection, firmware, partition layout
- **Fix** Bootloader, fstab, initramfs, drivers, network
- **Validate** QEMU smoke test → boot confirmed
- **Deploy** libvirt or Kubernetes (optional)

~5-15 minutes per VM • Deterministic • Batch ready

Windows VM Migration: Solving the Hard Problem

Windows VMs are the **hardest to migrate** — missing VirtIO drivers = immediate BSOD. hyper2kvm automates everything.

What hyper2kvm Does Automatically

Fix	Details
VirtIO Drivers	Injects viostor, netkvm, vioscsi, balloon, serial into DriverStore
Registry	Registers boot-critical drivers, sets start type, creates service entries
BCD Store	Updates boot configuration for VirtIO disk controller
Unattend.xml	Generates first-boot script for driver finalization
VMware Cleanup	Removes VMware Tools, PVSCSI refs, VMXNet3 config
Hyper-V Cleanup	Removes integration services, synthetic drivers
License	Extracts & preserves OEM/Retail/MAK/KMS keys
Active Directory	Detects domain membership, prepares rejoin

Supported Windows Versions

Windows 7 Windows 8/8.1 Windows 10 Windows 11
Server 2008 R2 Server 2012/R2 Server 2016
Server 2019 Server 2022 Server 2025

Migration Config

```
cmd: local
vmdk: /exports/win-server.vhdx
output_dir: /kvm
to_output: win-server.qcow2
out_format: qcow2
virtio_iso: /path/to/virtio-win.iso
# Auto-detects Windows, injects
# all required drivers & registry mods
```

Enterprise & Cloud-Native Deployment

hypersdk Enterprise Features

- **REST API** — 67+ endpoints, WebSocket updates
- **Job scheduling** — cron, dependencies, priority queue
- **RBAC** — admin, operator, viewer roles
- **Secrets management** — Vault, AWS SM, Azure KV
- **Audit logging** — full event trail
- **Rate limiting** — per-user and global
- **Prometheus metrics** — observability built-in
- **Cost estimation** — AWS S3, Azure Blob, GCS
- **Carbon-aware** — schedule by grid intensity
- **Web dashboard** — React-based monitoring

hyper2kvm Cloud-Native

- **One-command K8s deploy** — convert + upload + provision
- **KubeVirt** — VirtualMachine resource creation
- **OpenShift** — OperatorHub, SCC, Routes, OAuth
- **OLM bundle** — operator packaging
- **Batch processing** — parallel multi-VM migration
- **Daemon mode** — systemd service + REST API
- **LUKS/TPM2** — encryption support
- **Sandboxed LVM** — 100% host protection
- **TUI dashboard** — Textual/Curses/CLI
- **Debug bundles** — automated troubleshooting

```
# Migrate + deploy to K8s in one command
sudo h2kvmctl --config migration.yaml \
  --deploy-k8s --k8s-namespace prod \
  --k8s-vm-name web-01 --k8s-auto-start
```

hypersdk + hyper2kvm

"Export Anywhere, Migrate Once, Boot Right."

hypersdk (Go)

10 cloud providers, 67+ API endpoints, job scheduling, RBAC, audit

hyper2kvm
(Python)

480+ APIs, 35+ OS versions, deterministic fixes, first-boot success

Contract

JSON manifest — provider-agnostic, deterministic, resumable

Windows

Full VirtIO injection, registry mods, license & AD support

Cloud-Native

KVM, libvirt, KubeVirt, Kubernetes, OpenShift, k3s

Performance

5-7x faster, 66% less memory, minutes instead of hours

hypersdk: github.com/ssahani/hypersdk | **hyper2kvm:** github.com/ssahani/hyper2kvm | **PyPI:** pypi.org/project/hyper2kvm

License: LGPL-3.0